Current system validations

QUEUE2PLAY - a simple guide to the checks currently implemented in the Java program. Source reviewed on October 11, 2026.

Read this as a guide to current behavior, not a proposal for new rules. Examples describe expected outcomes from source inspection; they are not additional executed test results.

Input / check	Current rule
Full name	New registration: 5-60 characters; at least two letters; starts with a letter; letters, spaces and dots only.
Username / email	Username: 3-20 characters with at least one letter. Email: at most 100 characters and valid format.
Password / login	Password: 8-20 characters, no whitespace, control characters or |. Three failed authentications trigger a 30-second countdown.
Rental quantity	New orders: up to 3 copies of each game across that customer's pending requests and current cart; current cart quantity must fit available stock.
Duration / price	Rental duration: 1-30 days. Catalog rental price: $1.00-$10.00 inclusive.
Inventory	At most 20 total copies of a game, counting available copies and copies actively rented.
Older saved records	Compatibility rules differ: older account fields can be longer, and saved requests/rentals can contain up to 5 copies.

Where to look

02 Registration identity • 03 Password and login • 04 Menus, search and input • 05 Cart and order • 06 Queue and checkout • 07 Returns • 08 Inventory • 09 Saved accounts and games • 10 Saved rentals and queue • 11 Storage and verification.

Code reference: AccountManager, Queue2Play constants, GameCatalog, RentalManager, RentalDataStore and TerminalUI.

Registration identity

Surrounding spaces are trimmed from full name, username and email before these fields are checked.

Input / check	Current rule
Full name	5-60 Java string characters. Must begin with a letter and contain at least two letters. Only letters, ordinary spaces and dots are accepted. Digits, apostrophes, hyphens and other symbols are rejected.
Name examples	Juan Dela Cruz and J. Cruz pass. J fails the minimum length. Juan2 Cruz and Ana-Marie fail the allowed-character rule.
Name scope	The program does not require two separate words or verify a real identity. A single name such as Maria passes. Repeated spaces or dots are not separately prohibited.
Username	3-20 characters: ASCII letters A-Z/a-z, digits, dot, underscore or hyphen. At least one letter is required. Spaces and other symbols are rejected.
Username examples	ana, juan_23 and alex.ph pass if unused. ab is too short; 123 has no letter; juan cruz contains a space.
Unique identifiers	Existing usernames and emails are matched without regard to capitalization. An already used username or email is rejected.
Email format	At most 100 characters. Requires a local part, @, domain and a final suffix containing at least two ASCII letters. No spaces. Local characters allowed: ASCII letters, digits, . _ % + -.
Email dots / domain	The local part cannot start or end with a dot. No consecutive dots anywhere. Each domain label must contain 1-63 characters and cannot start or end with a hyphen.
Email examples	juan@gmail.com passes if unused. juan@gmail, .juan@gmail.com, juan..ph@gmail.com and juan@-mail.com fail. Format checking does not confirm that the mailbox exists.

Code reference: AccountManager.validRegistrationName, validRegistrationUsername, validRegistrationEmail and register.

Password and login

Passwords remain case-sensitive. Login accepts either the saved username or saved email, ignoring identifier capitalization.

Input / check	Current rule
New password	8-20 characters. Rejects |, whitespace, space characters and control characters. Letters only, digits only, a mixture, or permitted symbols are accepted. No uppercase/digit/symbol combination is required.
Password examples	abcdefgh, 12345678 and abc12345 pass. abc1234 is too short. abcd efgh, a tab, or abc|12345 fails. Passwords are not trimmed.
Confirmation	Confirmation must exactly equal the entered password, including capitalization. A mismatch shows: Passwords do not match. Please try again.
Empty login fields	Empty identifier: Username or email cannot be empty. Empty password: Password cannot be empty. These do not count as failed authentication attempts.
Wrong credentials	Unknown accounts and incorrect passwords show the same message: Incorrect username/email or password.
Attempt counter	Displays attempt 1/3, 2/3 and 3/3. Every third failed authentication triggers a 30-second countdown. Changing identifiers or leaving the login screen does not reset the running application's failure count.
Reset / navigation	Successful login clears the counter. Restarting starts a new counter. Navigation inputs do not count as failures. Login identifier accepts B/0 to return; password accepts B to return to identifier and C/0 to cancel.
Masked input / logs	Password input requires an interactive terminal; unsupported input raises an error. Attempts are logged without passwords. Control characters and | in failed-login identifiers are replaced with ? in the log.
Stored hash	PBKDF2-HMAC-SHA256 with a random 16-byte salt and 32-byte result. Stored hash parsing requires four parts, valid Base64 and 100,000-1,000,000 iterations. New hashes use 210,000 iterations. Malformed hashes do not authenticate.

Code reference: AccountManager.login, failedLogin, validRegistrationPassword; PasswordSecurity; TerminalUI.readPassword and showLoginCooldown.

Input, search and navigation

The screen validates choices for its current step. Keys shown on one screen are not automatically accepted on every other screen.

Input / check	Current rule
Text / numbers	Ordinary text input trims surrounding spaces. Whole-number fields use integer parsing: decimal quantities and out-of-range integers are rejected. Invalid input displays an error and requests another choice.
Yes / No	Confirmation prompts accept Y or YES and N or NO, without regard to capitalization. Other answers are retried. Checkout/return confirmations display ERROR: Use Y or N.
Discard confirmation	Yes abandons the unsaved transaction. No retains entered details and the current step. Editing registration or new-game details validates the edited field while preserving the other entered values.
Platform choice	Only the displayed choices for all games, PS5, Xbox or Nintendo are accepted. Stored platform values must use the exact supported names.
Catalog search	Blank search is rejected. Search ignores case: complete game ID, part of title, or complete platform name. It searches the entire selected platform list. Return-list search can be blank to restore the list.
Search / sorting	No match displays an empty result; it does not change inventory. Clear search retains sort. Search and sort changes return to page one. Sort cycles default, A-Z, Z-A, price ascending, price descending.
Displayed row number	Selection screens accept only a number from 1 through the number of games on the current page. Ordinary browsing does not select a game. Cart-line removal likewise requires an existing displayed line number.
Pagination	Only usable page keys are shown. Catalog pages have at most nine games and may show fewer to fit wrapped rows. Return lists explicitly reject moving beyond the first/last page.
Receipts / previews	Receipts and results use N/P. Batch confirmation previews use F/P so N can still mean No. Navigation and invalid input do not perform a checkout or return.
Terminal / input end	Configured terminal width is clamped to at least 40 and height to at least 20. End-of-input stops the input flow with InputEndedException rather than treating it as a valid choice.

Code reference: Queue2Play menu loops, paginateGames, selectActiveRentalsForReturn; TerminalUI readers, confirmations and pagination methods.

Cart and order checks

The three-copy limit is per customer, per game. It is not a limit of three different game titles in the cart.

Input / check	Current rule
Customer / game	Order submission requires a client account and a nonempty cart. Each selected game must still be the actual game stored in the catalog.
Active rental	A customer cannot add or submit a new order for a game they are already actively renting.
Repeated requests	The same game can be queued again while pending copies plus current cart copies stay at or below 3. Repeated selections inside the current cart combine into one line.
Quantity input	Positive whole number only. Maximum extra copies = the smaller of (3 - already queued - in cart) and (available stock - in cart). No extra copy is accepted when this maximum is zero.
Quantity examples	With 1 copy already queued and no cart copies, at most 2 more may be added if stock permits. With stock 1, requesting 2 in the current cart is rejected.
Available stock	Out-of-stock games cannot be added. The current cart must not request more copies than the game currently has available. Stock is checked again before submission.
Shared stock scope	Pending requests do not reserve or deduct stock. Different customers can each submit a request that individually fits available stock. Their combined pending requests may exceed that stock; later requests wait at checkout.
Duration	Choose preset 1, 3 or 7 days, or a custom whole number from 1 to 30 inclusive. Zero, negative values, 31 and decimal durations are rejected.
Final checks / IDs	Before creating requests, rechecks quantities 1-3, queued-plus-cart limit, active rentals, current catalog membership and available stock. Queue-number capacity is checked before creating all distinct game lines.
Confirmation / cost	Only confirmation submits the order. One request is created per distinct cart game. Pricing uses validated duration and quantity; VIP receives the existing 10% discount. Navigating or declining confirmation does not create a request.

Code reference: Queue2Play.validateRentalSelection, askQuantity, chooseRentalDays, editOrder, placeInQueue and validateCartBeforeSubmission; RentalManager.validateCart and calculateRentalCost.

Queue and checkout

Requests can wait even after a valid order was submitted, because available stock may change before administrator checkout.

Input / check	Current rule
No ready request	Approving when no request has sufficient stock displays a warning. It does not change stock, rentals or session counters.
Request still queued	Checkout rejects a null, removed or replaced request. The queue number must find the same request object, so an already processed request cannot be checked out again.
Request fields	Must have a client (not admin), a game, a positive queue number, quantity 1-5 under compatibility rules, and rental duration 1-30.
Current catalog / price	The request must refer to the current catalog game. Price must be finite and between $1.00 and $10.00. NaN, infinity and values outside the range are rejected.
Stock / active copies	Current available stock must cover the entire request quantity. Existing active copies plus this request must not exceed the saved-record compatibility limit of 5 per client/game.
Next / batch checks	The checkout screen selects the next eligible request. A batch uses predicted stock consumption. Before each batch item, it checks that the next eligible request is still that planned request; changes stop the batch.
VIP / waiting rules	VIP priority preserves arrival order within each membership group. After three VIP checkouts an eligible regular request gets a turn. Unavailable requests remain waiting while later eligible requests may proceed.
Customer cancellation	Input must parse as a queue number and locate a request owned by that customer. A different customer's request cannot be cancelled. Confirmation is required.
Admin cancellation	Input must parse as a queue number and locate an existing request. Cancellation requires confirmation. Declining does not remove it or add a cancellation slip.
Rental / commit guards	Rental number must be positive, below 2,147,483,647 and unique. Checkout validation runs before changes; successful checkout deducts stock and removes the request. Save failure restores the last committed state.

Current policy: the 5-copy checkout guard preserves old saved requests. It does not raise the 3-copy limit for new orders.

Code reference: CheckoutQueue.validRequest, eligibility/prediction methods; Queue2Play.checkoutManagement, processCheckoutBatch, cancelSessionRequest; RentalManager.validateCheckout and checkout.

Return checks

Administrators select displayed row numbers from the active return list, which is ordered by earliest due date.

Input / check	Current rule
Single / multiple rows	Accepts a displayed row number or comma-separated numbers such as 1,3,5. Every entry must parse as an integer and belong to the current page.
Duplicates / empty parts	The same row cannot be selected twice. An empty input, trailing comma, empty item, decimal, game ID or a row from another page is rejected.
Selection examples	1,3 passes if both rows exist. 1,1 fails duplicate selection. 1, fails integer parsing. 99 fails when it is outside the visible page.
Processing order	Selected rentals are processed in displayed due-date order, regardless of the order of typed numbers.
Active / not duplicated	The batch must be nonempty. Each rental must still have ACTIVE status and be present in the active rental collection. The same rental object cannot appear twice.
Return date	A return time must exist and cannot be earlier than checkout. The normal UI uses the current system time.
Stock restoration	Restored available stock cannot exceed 20. For several selected rentals of the same game, the combined restoration is checked before any rental is changed. Integer overflow is also checked.
Confirmation / retry	Confirmation is required before processing. Invalid answers are retried. No returns to the return selection flow without completing the selected rentals.
Late fee	Days late cannot be negative. On-time or early return means zero days late. Any overdue fraction of a day rounds up to a whole late day. Fee = days late x $1.00 x quantity.
Completed rental	Once completed, a rental leaves the active collection. Attempting to return it again is rejected. The selected return batch is saved through one recoverable commit.

Code reference: Queue2Play.parseReturnSelection, completeSelectedReturns and processReturn; RentalManager.completeReturn, daysOverdue and calculateLateFee.

Inventory checks

Adding, editing, updating stock and deleting games each validate their inputs before the change is committed.

Input / check	Current rule
Game title	Required, nonblank and at most 100 characters. No | or ISO control characters. Spaces, punctuation and digits are otherwise allowed. UI input trims surrounding spaces.
Duplicate game	The same title on the same platform is rejected without regard to title capitalization. The same title on a different supported platform is allowed. Editing title/platform checks the resulting combination.
Platform	Only PS5, Xbox and Nintendo are supported. UI selection must use a displayed platform choice.
Rental price	Must parse as a finite number from $1.00 to $10.00 inclusive. NaN, infinity, nonnumeric input, $0.99 and $10.01 are rejected. Input is not restricted to exactly two decimal places; displayed money is formatted.
New-game stock	Add-game UI requires a whole number from 1 to 20. Catalog/storage validation also permits 0 for an existing out-of-stock game.
Update quantity	Add/remove stock requires a positive whole number. Resulting available stock cannot be negative. Resulting available plus actively rented copies cannot exceed 20.
Stock examples	With 18 available and 2 actively rented, adding 1 fails the total-stock limit. Removing 19 from 18 available fails. Removing all 18 is allowed and leaves available stock at 0.
Delete restriction	A game with an active rental cannot be deleted. A game with pending requests can be deleted after confirmation; its pending requests are cancelled, and the warning states this.
Generated IDs	PS5 uses P, Xbox X and Nintendo N, followed by at least three digits. IDs must be unique in saved inventory. A positive numeric suffix must be below 2,147,483,647; generation stops when counter capacity is exhausted.
Confirmation / edit	New-game creation, stock changes and deletion require confirmation. Invalid edited fields are retried. Returning from an edit keeps the other previously entered details; discarding requires confirmation where the flow prompts for it.

Code reference: GameCatalog.validTitle, validId, validate, generateGameId and addGame; Queue2Play.addGame, updateGameStock and deleteGame; RentalManager.updatedStock.

Saved accounts and games

Saved-data checks protect loading and saving. Some limits intentionally remain broader so older valid files can still be read.

Input / check	Current rule
Account record	Exactly 6 pipe-separated fields. Username and email must be unique across login identifiers, ignoring case. Username and email cannot be identical. Role must be ADMIN/CLIENT; membership REGULAR/VIP.
Legacy username	3-32 ASCII letters, digits, dots, underscores or hyphens. Unlike new registration, a letter is not separately required.
Legacy full name	2-100 characters. Starts with a Unicode letter; remaining characters may be Unicode letters, combining marks, spaces, dots, apostrophes and hyphens.
Legacy email	At most 254 characters and the older email-format pattern. The stricter new-registration dot/domain checks are not applied to saved accounts.
Saved password	At least 8 characters and no ISO control characters. Legacy plaintext is converted to a salted hash in memory. A hash-prefixed value must have a valid hash structure. Saving requires hash-prefixed passwords.
Game record	Exactly 5 fields: ID, title, platform, price, available copies. Applies title/platform/price/stock validators. No duplicate game ID or duplicate title on the same platform.
Saved game ID	Matching uppercase P/X/N prefix plus 3-10 digits. Numeric suffix must be positive and below 2,147,483,647. ID prefix must agree with the platform.
Game counters	Counter header has 4 fields. Each platform counter is a nonnegative integer below 2,147,483,647. Existing IDs also advance counters to avoid reusing older numbers.
Total inventory	Available copies 0-20. Available plus all active rental copies of the same game must remain at or below 20.

Intentional startup policies retained

F1: built-in admin/alex/juan passwords are restored to their code-defined defaults at startup. F2: the first pre-upgrade .bak copy is preserved and may retain legacy plaintext. These are user-confirmed intentional behaviors, not changes introduced by this guide.

Code reference: AccountManager.loadAccounts and built-in account setup; User constructor; GameCatalog; RentalDataStore.loadGames and validateState; AppStorage.commit.

Saved rentals and queue

These checks reject records that refer to unknown clients, contain impossible values or conflict with active stock and queue state.

Input / check	Current rule
Rental schema / number	Accepts 9, 10 or 13 fields for old/current formats. Rental number must be unique, positive and below 2,147,483,647. Counter header has 2 fields and a positive integer counter.
Client / game / status	Client must exist and not be an admin. Status must be ACTIVE or COMPLETED. Active rentals require an existing game. Completed history may refer to a deleted game if its historical ID/title/platform are valid.
Dates / amount	Checkout/due dates must parse. Due must follow checkout and be no later than checkout + 31 days in the compatibility loader. New orders remain limited to 30 days. Rental amount must be finite and nonnegative.
Quantity / totals	Saved quantity is 1-5; old records without quantity default to 1. Active copies per client/game cannot exceed 5. Available plus active game copies cannot exceed 20.
Return fields	Return time, when present, cannot precede checkout. Days late and finite late fee must be nonnegative. ACTIVE records cannot have return details. No return date means no late days/fee. A recorded return requires COMPLETED status and exact matching computed late days/fee.
Queue schema / number	Accepts 4-6 fields. Unique positive queue number below 2,147,483,647; known nonadmin client and current game. Serialized VIP flag must be exactly true or false. Actual membership is taken from the client.
Queue days / copies	Duration 1-30; quantity 1-5. Missing old fields default to 3 days and 1 copy. Combined queued plus active copies per client/game cannot exceed 5. Saved waiting requests do not have to fit currently available stock.
Queue counters	Header has 2 or 3 fields. Next queue counter is positive. Saved VIP streak, when present, must be 0-3. Next numbers are advanced beyond loaded record numbers.
Before-save state	Unique rental/request numbers must be below their next counters. Active records must match the active collection and current game objects; completed rentals must not remain active. Requests must match current client membership and catalog objects.

Legacy date-only values are accepted: checkout becomes start of day; due becomes 11:59 PM. This compatibility conversion is not a new date-entry option.

Code reference: RentalDataStore.loadRentals, loadQueue, parseDate and validateState; CheckoutQueue.validRequest; RentalRequest defaults.

Storage checks and scope

A validation error should identify the problem instead of silently replacing saved business records with empty data.

Input / check	Current rule
Record parsing	Business files are read as UTF-8 with pipe-separated fields, including empty fields. Blank/comment lines are skipped except counter headers. Invalid account/game/rental/queue records report the file and line number and keep original data.
Missing files	Missing account files still allow built-in account creation; missing game files use starting games. Missing rental, queue and log files yield no records. A present malformed file is not treated as a missing file.
Log checks	Loaded log lines must have three fields, splitting at the first two | separators. ADMIN entries populate the audit log. New log timestamp/type cannot contain |; message cannot contain newline or carriage return. The loader does not separately validate timestamp format or restrict every log type.
Data-folder checks	Directory must be usable. Saved business paths must be regular files when present. Opening and committing use file locks; an overlapping access that cannot obtain the lock is rejected.
External changes	Before committing, compares all five saved business files to the bytes remembered by this session. If another editor/session changed a file, saving is rejected and the user is told to restart/reload.
Interrupted save	Known business files are written as one journalled set. An interrupted prepared transaction is recovered. Save failure restores the last committed application snapshot, rather than leaving only some in-memory business changes.
Capacity / overflow	Record IDs and counters are checked where created/loaded; cart submission checks queue-number space before mutation. Stock and return calculations use widened or checked arithmetic to reject overflow.

How this guide was verified

Reviewed the current Java validators, input loops, transaction prechecks and file loaders. Created this PDF, extracted its text, checked page bounds and rendered every page for visual inspection. No application logic or production data was changed. Application tests were not rerun for this documentation-only request.

Examples are illustrative, not PASS/FAIL test records. This guide documents checks visible in the reviewed source; it does not claim real-world identity, email ownership, or every possible corrupted input is verified.

Code reference: RentalDataStore.records, invalidRecord, writeLogRecord, loadLogs, saveData and Snapshot; AppStorage startup, commit and recovery; all preceding code references.